ARTIFICIAL INTELLIGENCE · CO1 ASSIGNMENT

Design, Analyse and Validate an Efficient Search Strategy
for a Warehouse Picking Robot

Course Code: CSA17 | B.E./B.Tech | CO1 | Total: 50 Marks

Name: Richard Benson Raj.B.A

Register Number: 1925260077

Submission Date: 06-11-2026

Assignment answers include all 12 tasks, Python implementations,
complexity analysis, performance evidence templates, and Day-4 challenge.

TASK 1 – Design the Agent and Search Algorithms (5 Marks)

1.1 PEAS description

PEAS element	Warehouse picking robot
Performance measure	Reach requested shelf/dock safely; minimize travel time, distance, collisions, energy, and order delay; maintain reliable rou
Environment	Warehouse aisles, junctions, shelves, packing stations, charging dock, staff and other robots.
Actuators	Drive motors, steering, brakes, lift/rack mechanism, status display, route communication.
Sensors	QR/floor-marker reader, wheel encoder, lidar/camera, proximity sensors, battery sensor, map and traffic updates.

1.2 Environment classification

Property	Classification and reason
Observable	Initially fully observable under working QR/localization and a known map; temporary occupancy can still require updates.
Deterministic	Assume moves follow the graph transition model. Real traffic may introduce uncertainty, so this is an idealized model.
Sequential	Each move affects the robot's later location and route choices.
Dynamic	Staff and robots can occupy aisles while the robot is planning.
Discrete	Locations and actions are represented as graph nodes and edges; time/cost can be numeric.
Multi-agent	Other robots and staff share the warehouse and affect available paths.

AI view: Acting rationally. The robot selects actions expected to achieve the goal with low cost using its percepts and model. It is a rational problem-solving agent because it searches for a route and chooses actions that maximize goal achievement/performance.

Historical connection: Early AI problem solving represented tasks as states, operators and goals, and used search to explore possible action sequences. Warehouse routing is a modern graph-search problem of the same general type.

1.3 Problem formulation

• State: current warehouse junction/location (and, if needed, map/traffic status).

• Initial state: robot charging dock S.

• Actions: move to any adjacent accessible location.

• Transition model: Result(s,a)=the adjacent node reached after action a.

• Goal test: current location equals requested shelf bay G.

• Path cost: sum of travel-time edge costs. For the uninformed algorithms below, each edge is treated as one step when finding minimum-hop paths; weighted travel time is reported separately. BFS is optimal only for equal edge costs.

1.4 Three distinct uninformed strategies

A. Breadth-First Search (BFS) – iterative queue

BFS expands nodes level by level. It is appropriate for unweighted/equal-cost routes and finds a minimum number of edges. It uses a FIFO queue and a visited set.

```
BFS(start, goal):
    queue = [start]
    parent[start] = None
    visited = {start}
    while queue is not empty:
        u = queue.pop_front()
        if u == goal: return reconstruct(parent, goal)
        for v in neighbors(u):
            if v not in visited:
                visited.add(v); parent[v] = u
                queue.push_back(v)
    return FAILURE
```

B. Depth-Limited Search (DLS) – recursive

DLS explores depth-first but stops at a chosen limit L. It is memory-efficient and useful when a reasonable route-depth bound is known, but it may miss a goal deeper than L and can revisit nodes in cyclic graphs unless path checking is used.

```
DLS(node, goal, limit, path):
    if node == goal: return path
    if limit == 0: return CUTOFF
    cutoff_seen = False
    for child in neighbors(node):
        if child not in path:
            result = DLS(child, goal, limit-1, path + [child])
            if result is a route: return result
            if result == CUTOFF: cutoff_seen = True
    if cutoff_seen: return CUTOFF
    return FAILURE
```

C. Iterative Deepening Depth-First Search (IDDFS) – repeated recursive DLS

IDDFS runs DLS with limits 0,1,2,… until a goal is found. It combines BFS's shallow-solution guarantee (for unit costs) with DFS's low memory. It is a distinct strategy because it repeatedly searches at increasing depth limits.

```
IDDFS(start, goal):
    for limit = 0, 1, 2, ...:
        result = DLS(start, goal, limit, [start])
        if result is a route: return result
        if result == FAILURE: return FAILURE
```

Preprocessing: Convert the warehouse map to adjacency lists in O(V+E) time and space. BFS needs a visited set and parent map. DLS needs a current-path set/list to prevent cycles. IDDFS uses path checking per iteration; it does not retain a global visited set across depth limits because that could prune valid shallow paths.

Notation: V = number of locations, E = edges, b = branching factor, d = shallowest goal depth, m = maximum depth, C* = optimal path cost, ε = minimum positive step cost.

TASK 2 – Identify the Basic Operation (2 Marks)

Algorithm	Basic operation and reason
BFS	Node expansion: remove one node from queue and generate its successors. Each expansion examines adjacent edges; gro
DLS	Recursive node expansion / successor generation. It is repeated for each node on each depth-first branch; depth limit contro
IDDFS	Recursive node expansion across all depth-limit iterations. Nodes near the root are re-expanded many times, which causes

These operations are suitable because they dominate the number of graph locations and transitions processed. Goal tests and queue/stack operations are O(1) per node in typical implementations.

TASK 3 – Derive Time and Space Complexity (6 Marks)

3.1 BFS

At depth i, a tree with branching factor b has at most b^i nodes. Expansions through depth d are:

$N = 1 + b + b^2 + \dots + b^d = (b^{(d+1)}-1)/(b-1)$, for $b \neq 1$.

Each expanded node generates up to b successors, so $T_{BFS} = \Theta(\Sigma_{i=0 \text{ to } d}\, b^i) = \Theta(b^d)$ for b>1. The queue may contain a full level: $S_{BFS} = \Theta(b^d)$. In a graph with duplicate detection, each vertex is visited at most once and edge scanning gives O(V+E), with O(V) visited/parent memory.

Including preprocessing: $T_{total} = O(V+E) + O(V+E) = O(V+E)$; $S_{total} = O(V+E)$ for adjacency lists plus O(V) search metadata. If the graph is generated implicitly as a tree, use the exponential bounds above.

3.2 DLS

At limit L, the maximum number of recursive calls is:

$N_{DLS}(L) = \Sigma_{i=0 \text{ to } L}\, b^i = (b^{(L+1)}-1)/(b-1) = \Theta(b^L)$, b>1.

Time $T_{DLS}=\Theta(b^L)$ in the worst case. Recursive stack and current path use $S_{DLS}=O(L)$ (excluding stored graph). With adjacency-list preprocessing, total time $O(V+E+b^L)$, and memory $O(V+E+L)$.

3.3 IDDFS

At depth limit ■, nodes at depth i are visited once for each limit ■≥i. Total calls through limit d:

$N_{IDDFS} = \Sigma_{i=0 \text{ to } d}\, (d-i+1)b^i = (d+1) + d\,b + (d-1)b^2 + \dots + b^d$.

For b>1, the deepest level dominates, so T_IDDFS=Θ(b^d), with a constant-factor re-expansion overhead (approximately b/(b–1) relative to BFS in a full tree). Space is O(d) for the recursive stack/path, excluding graph storage. Including preprocessing: O(V+E+b^d) time and O(V+E+d) memory.

For weighted paths, uninformed BFS/DLS/IDDFS do not guarantee minimum travel-time cost unless every edge cost is equal. Uniform-cost search would be needed for arbitrary positive costs, but it is not one of the three selected uninformed strategies.

TASK 4 – Best, Average and Worst Cases (4 Marks)

Algorithm	Best case	Average case	Worst case
BFS	Goal is start (1 expansion), or first nodes dequeued	Goal is at shallowest level among nodes; roughly half of nodes at the goal level	Only the deepest nodes at the goal level remain before the goal is found
DLS	Goal at root or first branch within limit	Depends strongly on branch ordering and goal depth; may search full depth-limited tree before finding goal	Goal beyond limit or absent; full depth-limited tree explored
IDDFS	Goal at depth 0; little repetition.	Goal at depth d; repeated shallow expansions, but still roughly linear in nodes up to depth d	Maximum depth limit; most work at deepest level

In a finite warehouse graph, use a maximum depth/visited-state policy to avoid endless IDDFS on cycles. Average-case counts depend on goal distribution, graph connectivity and neighbor ordering; no single average count is valid without those assumptions.

TASK 5 – Asymptotic Analysis (3 Marks)

Strategy	Tree-model time	Tree-model space	Graph with duplicate detection
BFS	O(b^d), Θ(b^d) worst	O(b^d)	O(V+E) time, O(V) metadata
DLS	O(b^L)	O(L) stack/path	May revisit nodes; bounded depth
IDDFS	O(b^d) (re-expansion constant)	O(d) stack/path	Repeated work; depth limit needed

Increasing states from 1,000 to 1,000,000 is a 1,000× increase in V. A graph-search BFS with O(V+E) work may remain practical if the graph is sparse, but its visited/parent memory grows linearly. DLS/IDDFS keep search-stack memory small, yet may spend exponential time exploring branches. For a large warehouse, memory can become BFS's first bottleneck; depth-first methods trade memory for repeated or potentially wasted search.

TASK 6 – Recurrence Relation (3 Marks)

For recursive DLS on a full b-ary tree with remaining depth n, each call performs constant local work and may call DLS on b children with depth n–1.

T(n) = bT(n–1) + c, for n>0; T(0)=c■.

Here n is the remaining depth limit; each child subproblem has size n–1; there are b recursive calls; outside-call work c includes goal test, limit check and loop overhead (constant per node, excluding edge generation if degree varies).

TASK 7 – Solve the Recurrence (4 Marks)

Use iteration/substitution. T(n)=bT(n–1)+c:

T(n)=b[bT(n–2)+c]+c = b²T(n–2)+c(b+1).

After k steps: T(n)=b^kT(n–k)+c(1+b+…+b^(k–1)). Set k=n:

T(n)=b^nT(0)+c(b^n–1)/(b–1) = Θ(b^n), for b>1.

This method applies because the recurrence has the same branching factor at each depth and constant local work. As b or n grows, calls increase exponentially. IDDFS repeats depth-limited searches, giving Σ(i=0..d)(d–i+1)b^i = Θ(b^d); repeated shallow work increases the constant but not the exponential order.

TASK 8 – Implement and Test Algorithms (4 Marks)

The following Python 3 code implements BFS, recursive DLS and IDDFS. It also returns operation metrics. It can be used in Jupyter/Colab.

```
from collections import deque
from time import perf_counter
```

```
def bfs(graph, start, goal):
    q = deque([start]); parent = {start: None}
    generated = 1; expanded = 0; max_frontier = 1
    while q:
        u = q.popleft(); expanded += 1
        if u == goal:
            return route(parent, goal), generated, expanded, max_frontier
        for v in graph.get(u, []):
            if v not in parent:
                parent[v] = u; q.append(v); generated += 1
        max_frontier = max(max_frontier, len(q))
    return None, generated, expanded, max_frontier

def route(parent, goal):
    out = []
    while goal is not None:
        out.append(goal); goal = parent[goal]
    return out[::-1]

def dls(graph, start, goal, limit):
    expanded = 0; generated = 1; max_frontier = 1
    def rec(u, depth, path):
        nonlocal expanded, generated, max_frontier
        expanded += 1
        if u == goal: return path
        if depth == limit: return None
        for v in graph.get(u, []):
            if v not in path:
                generated += 1
                max_frontier = max(max_frontier, depth+2)
                ans = rec(v, depth+1, path+[v])
                if ans is not None: return ans
        return None
    ans = rec(start, 0, [start])
    return ans, generated, expanded, max_frontier

def iddfs(graph, start, goal, max_depth):
    total_gen = total_exp = 0; max_frontier = 1
    for limit in range(max_depth+1):
        ans, gen, exp, frontier = dls(graph, start, goal, limit)
        total_gen += gen; total_exp += exp
        max_frontier = max(max_frontier, frontier)
        if ans is not None:
            return ans, total_gen, total_exp, max_frontier
    return None, total_gen, total_exp, max_frontier

# Small reproducible test graph
G = {0:[1,2], 1:[3,4], 2:[5], 3:[], 4:[5], 5:[]}
for name, fn in [('BFS', lambda: bfs(G,0,5)),
                 ('DLS', lambda: dls(G,0,5,4)),
                 ('IDDFS', lambda: iddfs(G,0,5,5))]:
    t0 = perf_counter(); result = fn(); elapsed = perf_counter()-t0
    print(name, result, round(elapsed,6))
```

Test input: graph G above, start=0, goal=5. Expected: all algorithms return a valid route; BFS returns a shortest-hop route [0,2,5]. DLS/IDDFS return a valid route depending on neighbor order. For start=goal, expected route [0]. For unreachable goal (e.g., goal=99), expected None.

Important: the assignment requests 1k, 10k, 100k and 1M graph experiments. Use the benchmark section below to generate those sizes. A million-node graph may require substantial RAM; run it on a machine that can accommodate adjacency lists and metadata.

TASK 9 – Experimental Performance (3 Marks)

Use a sparse chain-plus-shortcuts graph generator to test scaling consistently. Set and report the random seed. Run each algorithm at least five times and average elapsed time. Node counts and frontier size are more reliable than timing.

```
import random, statistics, time

def make_graph(n, seed=17):
    random.seed(seed)
    g = {i: [] for i in range(n)}
    # Chain ensures reachability from 0 to n-1
    for i in range(n-1):
        g[i].append(i+1)
    # Add sparse forward shortcuts (acyclic graph)
    for i in range(n):
        for _ in range(2):
            j = random.randint(i+1, n-1) if i+1 < n else i
            if j != i and j not in g[i]: g[i].append(j)
    return g

def benchmark(n, runs=5):
```

```
    g = make_graph(n, seed=17)
    results = {}
    # BFS and IDDFS to depth 30 are feasible examples; set max_depth
    # according to your graph and computer for DLS/IDDFS.
    for name, fn in [
        ('BFS', lambda: bfs(g,0,n-1)),
        ('DLS', lambda: dls(g,0,n-1,n-1)),
        ('IDDFS', lambda: iddfs(g,0,n-1,n-1))]:
        times=[]; last=None
        for _ in range(runs):
            t=time.perf_counter(); last=fn(); times.append(time.perf_counter()-t)
        results[name]=(statistics.mean(times), last[1], last[2], last[3],
                       len(last[0])-1 if last[0] else None)
    return results

# Start with sizes [1000, 10000]. Add 100000 and 1000000
# only after confirming available RAM and acceptable runtime.
for n in [1000, 10000]:
    print(n, benchmark(n))
```

Record each size in a table with columns: N, algorithm, mean seconds (5 runs), generated, expanded, maximum frontier, solution depth, path cost, optimal? For this graph all edges have unit cost, so BFS is optimal. DLS/IDDFS may find non-shortest routes depending on order.

N (states)	Algorithm	Mean time (s)	Generated	Expanded	Max frontier	Depth
1,000	BFS	Run code	Run code	Run code	Run code	Run code
10,000	BFS	Run code	Run code	Run code	Run code	Run code
100,000	BFS	Run code	Run code	Run code	Run code	Run code
1,000,000	BFS	Run code	Run code	Run code	Run code	Run code
Repeat rows for	DLS and IDDFS					

Do not invent measured values. Paste the actual program output into this table and create log-scale plots (N vs time, generated nodes and max frontier).

TASK 10 – Compare Theory with Experiment (4 Marks)

• BFS should expand each reachable vertex at most once in a graph with a parent/visited dictionary; generated and expanded counts should rise approximately with V+E.

• On a sparse graph, BFS frontier memory is related to the widest explored layer. A chain-like graph has a small frontier, while a wide layered graph can have a much larger frontier.

• DLS has a small recursion/path memory, but can explore long branches and may fail if its limit is too small. Cycles are avoided using the current path.

• IDDFS re-expands shallow nodes at every depth limit; its expanded count should exceed DLS/BFS for the same depth, although the deepest layer dominates in a tree.

• Timing may deviate from theoretical growth because of Python overhead, cache behavior, graph shape, neighbor order, random seed and available memory.

TASK 11 – Recommend and Defend the Final Design (4 Marks)

For the original fully localized warehouse, recommend BFS as the baseline route planner only when each move has equal cost, because it is complete on a finite graph, finds a minimum-hop route, and graph duplicate detection bounds work by O(V+E). It is easy to validate and maintain.

If edge travel times differ, BFS is not guaranteed to minimize travel time. The assignment limits the investigation to uninformed methods, so report this limitation and recommend a later weighted-cost planner (Uniform-Cost Search) for production deployment.

DLS is useful for memory-limited exploration with a known depth bound, but it may miss the target. IDDFS is preferable when memory is very limited and a shallow solution is expected, at the cost of repeated expansions. Preprocessing adjacency lists costs O(V+E) and is reusable across many orders; dynamic aisle blockages require refreshing or temporarily filtering edges.

Final choice must be confirmed with your own benchmark metrics. For large sparse graphs, BFS's O(V+E) graph-search performance is often more predictable than exponential tree-search bounds, but its O(V) metadata can consume memory.

TASK 12 – Day-4 Design Challenge: Robot Location Is Uncertain (8 Marks)

A. Impact analysis (2 marks)

The robot no longer knows one exact start node. It knows a set of k possible junctions. The fully observable and known-start assumptions are violated. Its state must include its current belief (set of possible locations), not only a single physical location.

B. Evaluate original design (2 marks)

Ordinary BFS can still be used for a single known node, but running it from one guessed node is unsafe: it may produce a route that fails from the actual position. A belief state is $B \subseteq V$. In the worst case, the number of possible belief states is 2^V, so naïve belief-state search can be much larger than the original V-node graph.

C. Adaptation / redesign (2 marks)

Use a belief-state planner with local sensing. For each action, update the set of possible positions by applying the transition model to every location in B; then remove locations inconsistent with sensor observations. If the robot has a local QR/lidar sensor, it can often identify its location and collapse the belief set to a singleton. A practical conditional plan is: move toward a safe sensing point, localize, then run BFS from the observed location to the dock.

```
def belief_successors(graph, belief, action):
    # action maps each possible location to its next location.
    # Keep all possible next locations; a real robot also filters
    # them using the observation received from its sensor.
    return frozenset(action[u] for u in belief if u in action)

def sensor_update(belief, observation, sensor_model):
    # sensor_model(location) returns the sensor label predicted
    # at that location. Keep states matching actual observation.
    return frozenset(u for u in belief
                     if sensor_model(u) == observation)
```

D. Small evidence experiment (2 marks)

Illustrative 1-D warehouse: locations 0,1,2,…,10; dock=0. Possible start positions are {2,…,k+1}. Without a sensor, a conservative plan must handle all possible starts; with a perfect local sensor, the robot identifies its exact start and then uses BFS. The following code computes belief sizes and worst-case route lengths for k=2,4,8.

```
def belief_experiment(k):
    # Possible starts are k consecutive nodes; dock at 0.
    belief = set(range(2, 2+k))
    worst_route = max(belief)  # chain distance to dock
    no_sensor_belief_states = 2**len(belief)
    with_sensor_belief_size = 1
    return (k, len(belief), no_sensor_belief_states,
            with_sensor_belief_size, worst_route)

for k in [2,4,8]:
    print(belief_experiment(k))
```

k	Initial belief size	Possible subsets (2^k)	After perfect sensor	Worst-case route length
2	2	4	1	3
4	4	16	1	5
8	8	256	1	9

Expected interpretation: belief uncertainty grows exponentially in the number of possible locations if all subsets are represented. A perfect local sensor reduces the belief to one location, after which ordinary BFS is used. These values are a small theoretical demonstration; for submission evidence, run the code and include its actual output. A noisy sensor requires probabilistic beliefs and repeated observations rather than an immediate singleton.

Conclusion

The warehouse robot is a rational, goal-directed problem-solving agent. BFS, DLS and IDDFS represent different trade-offs between completeness, route depth, time and memory. For a known start and sparse finite graph, BFS is the clearest baseline; for uncertain localization, belief-state tracking plus local sensing is safer than blindly reusing a single-start route. The final recommendation should be supported by the student's measured benchmark output.

References

• Russell, S. and Norvig, P., Artificial Intelligence: A Modern Approach, 4th Edition, Pearson.

• Python 3 documentation: collections.deque, time.perf_counter.
• Course assignment handout, CSA17 Artificial Intelligence, CO1, 2026.